

Lecture 10: Foldables and Traversables

COMP2221: Functional programming

Maximilien Gadouleau

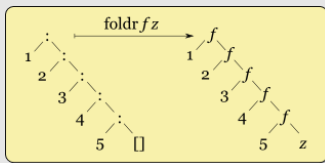
`m.r.gadouleau@durham.ac.uk`

**Folds: (yet another) family of
higher order functions**

- *folds* process a data structure in some order and build a return value
- Haskell provides a number of these in the standard prelude, with more available in the `Data.List` module

`foldr`: right associative fold

```
foldr :: (a -> b -> b) -> b -> [a] -> b  
foldr f z []      = z  
foldr f z (x:xs) = x `f` (foldr f z xs)
```

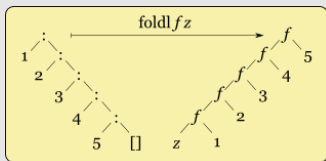


Folds

- *folds* process a data structure in some order and build a return value
- Haskell provides a number of these in the standard prelude, with more available in the `Data.List` module

`foldl`: left associative fold

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f z []      = z
foldl f z (x:xs) = foldl f (z `f` x) xs -- tail recursive!
```



How to think about this

- `foldr` and `foldl` are recursive
- However, often easier to think of them *non-recursively*

foldr

Replace `(:)` by the given function, and `[]` by given value.

```
sum [1, 2, 3]
= foldr (+) 0 [1, 2, 3]
= foldr (+) 0 (1:(2:(3:[])))
= 1 + (2 + (3 + 0))
= 6
```

foldl

Same idea, but associating to the left

```
sum [1, 2, 3]
= foldl (+) 0 [1, 2, 3]
= foldl (+) 0 (1:(2:(3:[])))
= (((0 + 1) + 2) + 3)
= 6
```

Purpose of folds

- Capture many linear recursive patterns in a clean way
- Can have efficient library implementation $\Rightarrow$ can apply program optimisations
- Actually apply to all `Foldable` types, not just lists
- e.g. `foldr`'s type is actually
$$\text{foldr} :: \text{Foldable } t \Rightarrow (a \rightarrow b \rightarrow b) \rightarrow b \rightarrow t\ a \rightarrow b$$
- So we can write code for lists and (say) trees identically

Folds are general

- Many library functions on lists are written *using folds*
$$\begin{aligned}\text{product} &= \text{foldr } (*)\ 1 \\ \text{sum} &= \text{foldr } (+)\ 0 \\ \text{maximum} &= \text{foldr1 } \max\end{aligned}$$
- Practicals ask you to define some others

Which to choose?

foldr

- Generally `foldr` is the right choice
- Works even for infinite lists
- Note `foldr (:) [] == id`
- Can terminate early

foldl

- Can't terminate early
- Doesn't work on infinite lists
- Usually best to use *strict* version:

```
import Data.List
foldl' -- note trailing '
```

- Aside: it is probably a historical accident that `foldl` is not strict (see <http://www.well-typed.com/blog/90/>)

⇒ CAUTION: `foldr` and `foldl` lead to different results if the operator `f` is not commutative

Foldable data structures

- **Foldable** type class: if we can *combine* an **a** and a **b** to produce a new **b**, then, given a start value and a container of **as** we can reduce it to a **b**

```
class Foldable f where
  -- minimal definition requires this
  foldr :: (a -> b -> b) -> b -> f a -> b

data List a = Nil | Cons a (List a)
  deriving (Eq, Show)

instance Foldable List where
  foldr :: (a -> b -> b) -> b -> List a -> b
  foldr _ z Nil = z
  foldr binop z (Cons a tail) = a `binop` (foldList binop z tail)
```


Traversables

Traversing a list

Functors (applicatives, monads, etc.) are used to encapsulate an object of a certain type. But how would you encapsulate a whole list of objects? Haskell uses `sequenceA`.

```
instance Traversable [] where
  -- sequenceA :: Applicative f => [f a] -> f [a]
  sequenceA []      = pure []
  sequenceA (u:us) = (:) <$> u <*> sequenceA us

sequenceA :: Applicative f => t (f a) -> f (t a)
```

`sequenceA` evaluates each action in the structure from left to right, and collects the results.

```
Prelude> sequenceA [Just 1, Just 2, Just 3]
Just [1,2,3]
Prelude> sequenceA [Right 1, Right 2, Right 3]
Right [1,2,3]
```

Traversing a list

What if we first wanted to apply a function to all the elements of the list? For that, Haskell uses the `traverse` function.

```
traverse :: Applicative f => (a -> f b) -> t a -> f (t b)
```

The `traverse` function maps each element of a structure to an action, evaluates these actions from left to right, and collects the results.

```
Prelude> traverse Just [1,2,3,4]
Just [1,2,3,4]
```

```
Prelude> traverse id [Right 1, Right 2, Right 3, Right 4]
Right [1,2,3,4]
```

```
instance Traversable [] where
  traverse _ []      = pure []
  traverse f (x:xs) = (:) <$> f x <*> traverse f xs
```

Relationships between `sequenceA` and `traverse`

We have

```
traverse :: Applicative f => (a -> f b) -> t a -> f (t b)
traverse f = sequenceA . fmap f
```

and

```
sequenceA :: Applicative f => t (f a) -> f (t a)
sequenceA = traverse id
```

The class `Traversable` consists of all types that support `traverse` (or equivalently, `sequenceA`). Once again, there are laws that `traverse` and `sequenceA` must satisfy, but we will not look into them – the Prelude lists those laws, if you are interested.